

MARAN SUITE SYSTEM — Especificaciones de Desarrollo: Módulo SPA

Fecha: Marzo 2026

Módulo: SPA

Prioridad: Alta — Cierre operativo antes de migración a PostgreSQL

Contexto general

El módulo SPA ya tiene estructura base funcional: cabinas, categorías, tratamientos, agenda con vista diaria y cuentas/folios básicos. Las siguientes tareas completan el módulo a nivel operativo real.

Las tareas están ordenadas de menor a mayor complejidad. Se recomienda ejecutarlas en orden para evitar dependencias rotas.

TAREA 1 — Validación de superposición de turnos en el mismo gabinete

Prioridad: Alta | **Complejidad:** Baja | **Área:** Backend

Descripción

Al crear o editar un turno (`spaAppointments`), el backend debe verificar que el gabinete (`cabinId`) no tenga otro turno activo que se superponga en el mismo horario. Si hay superposición, debe rechazar la operación con un mensaje claro.

Lógica de validación

Un turno ocupa desde `startTime` hasta `startTime + duration` (en minutos).

Hay superposición si:

```
nuevoInicio < existenteFin AND nuevoFin > existenteInicio
```

Solo considerar turnos con estado: `scheduled`, `confirmed`, `in_progress`.

Excluir el propio turno al editar (por `id`).

Cambios requeridos

`server/routes.ts` — **POST `/api/spa/appointments` y PATCH `/api/spa/appointments/:id`**

Antes de insertar o actualizar, ejecutar la consulta de superposición.

Si hay conflicto, retornar:

```
{
  "error": "Superposición de turno",
  "message": "El gabinete 'Cabina 1' ya tiene un turno de 10:00 a 11:00 hs. Elegí
```

HTTP status: 409 Conflict

`client/src/pages/spa.tsx` — **Formulario de turno**

Mostrar el mensaje de error del servidor en un toast o alerta visible al usuario.

No cerrar el modal si hay error de superposición.

TAREA 2 — Editar y cancelar turnos con confirmación

Prioridad: Alta | **Complejidad:** Baja | **Área:** Frontend

Descripción

Actualmente los turnos de la agenda no permiten edición ni cancelación desde la interfaz. Agregar estas acciones con confirmación antes de cancelar.

Cambios requeridos

`client/src/pages/spa.tsx` — **Vista de agenda / slots de turno**

Al hacer clic en un turno ocupado, mostrar un menú o modal con las opciones:

- **Editar turno** → Abre el formulario de turno precargado con los datos actuales. Al guardar, hace PATCH al endpoint existente.
- **Cancelar turno** → Muestra un dialog de confirmación con el texto:
"¿Estás seguro que querés cancelar el turno de [Nombre del cliente] el [fecha] a las [hora] en [Cabina]? Esta acción no se puede deshacer."
Botones: **"Sí, cancelar turno"** (destructivo, rojo) y **"Volver"**.

Si confirma, hace PATCH al endpoint con `status: "cancelled"`.

Estados visuales

Los turnos cancelados deben mostrarse visualmente diferenciados (por ejemplo, texto tachado o color gris) si se decide mostrarlos en la grilla. Alternativamente, ocultarlos de la vista activa pero mantenerlos en base de datos.

TAREA 3 — Inventario unificado con soporte de áreas

Prioridad: Media | **Complejidad:** Media | **Área:** Backend + Frontend

Descripción

El inventario actual (`inventoryItems`) es genérico. Se necesita que cada ítem pueda pertenecer a un área específica del hotel (SPA, Restaurante, Housekeeping, Mantenimiento, Administración, General) para filtrar y gestionar stock por sector.

Cambios en el schema (`shared/schema.ts`)

Agregar campo `area` a la tabla `itemCategories`:

```
area: text("area").notNull().default("general"),
// Valores posibles: "spa", "restaurant", "housekeeping", "maintenance", "admin",
```

Agregar campo `area` a la tabla `inventoryItems` como campo derivado o directo (puede heredarse de la categoría o setearse por ítem).

Cambios en el backend (`server/routes.ts`)

GET /api/inventory/items — Aceptar query param `?area=spa` para filtrar por área.

GET /api/inventory/categories — Aceptar query param `?area=spa` para filtrar categorías por área.

Cambios en el frontend

`client/src/pages/inventory.tsx`

- Agregar selector de área en la vista principal (tabs o dropdown): Todos / SPA / Restaurante / Housekeeping / Mantenimiento / Administración.
- Al crear/editar una categoría, mostrar campo "Área" con dropdown de las opciones.
- Al crear/editar un ítem, mostrar el área de su categoría como referencia (solo lectura)

o editable).

`client/src/pages/spa.tsx` — Sección de inventario/insumos del SPA

- Agregar una tab o sección "Insumos" dentro del módulo SPA que muestre los ítems de inventario filtrados por `area = "spa"`.
- Permitir ver stock actual y agregar movimientos de salida desde el SPA (por ejemplo, productos usados en un tratamiento).

TAREA 4 — Planning semanal por gabinete

Prioridad: Media | **Complejidad:** Media | **Área:** Frontend

Descripción

Agregar una vista de planning semanal (7 días) al módulo SPA, similar al planning del hotel, que muestre los gabinetes como filas y los días como columnas, con resumen de turnos por día.

Referencia

El componente `client/src/pages/planning.tsx` tiene la grilla base. Reutilizar la lógica y adaptar para SPA.

Cambios requeridos

`client/src/pages/spa.tsx`

Agregar toggle de vista en la sección Agenda: **"Vista diaria"** | **"Vista semanal"**

Vista semanal — comportamiento:

- Filas: una por gabinete/cabina activa.
- Columnas: 7 días a partir de la fecha seleccionada (navegable con flechas anterior/siguiente semana).
- Cada celda muestra: cantidad de turnos reservados en ese día para ese gabinete, con un color de fondo según ocupación:
 - 0 turnos → blanco/gris claro
 - 1-2 turnos → verde claro
 - 3-4 turnos → ámbar

- 5+ turnos → rojo (alta demanda)
- Al hacer clic en una celda, cambia a la vista diaria de ese gabinete en esa fecha.

Nuevo endpoint necesario en `server/routes.ts` :

```
GET /api/spa/appointments/weekly-summary?startDate=YYYY-MM-DD&endDate=YYYY-MM-DD
```

Retorna: array de `{ cabinId, date, count }` para el rango de fechas pedido.

TAREA 5 — Folio completo de turno: cargos, pagos anticipados y cierre tipo restaurante

Prioridad: Alta | **Complejidad:** Alta | **Área:** Backend + Frontend

Descripción

Esta es la tarea más importante del módulo. El cierre de turno actual es básico. Debe reemplazarse por un folio completo similar al cierre de mesa del restaurante, con las siguientes capacidades:

1. Agregar cargos más allá del tratamiento (productos, consumos, extras como pileta)
 2. Registrar pagos anticipados (señas) que descuenten del total
 3. Cerrar con múltiples métodos de pago
 4. Cargar a habitación del hotel (integración con folio de reserva)
 5. Emitir comprobante (Ticket, Factura A/B/C)
-

5.1 — Cambios en el schema (`shared/schema.ts`)

Tabla `spaAccountItems` — agregar campo `itemType` :

```
itemType: text("item_type").notNull().default("treatment"),  
// Valores: "treatment", "product", "extra", "service"
```

Nueva tabla `spaPayments` :

```
export const spaPayments = pgTable("spa_payments", {  
  id: serial("id").primaryKey(),
```

```

accountId: integer("account_id").notNull().references(() => spaAccounts.id),
amount: numeric("amount", { precision: 10, scale: 2 }).notNull(),
method: text("method").notNull(),
// Métodos: "cash", "debit_card", "credit_card", "transfer", "mercadopago", "ro
isAdvance: boolean("is_advance").default(false), // true = seña/anticipo
appointmentId: integer("appointment_id").references(() => spaAppointments.id),
// Si method = "room_charge", guardar referencia a reserva:
reservationId: integer("reservation_id").references(() => reservations.id),
notes: text("notes"),
createdAt: timestamp("created_at").defaultNow(),
});

```

Tabla spaAccounts — agregar campos:

```

receiptType: text("receipt_type"), // "ticket", "factura_a", "factura_b", "factur
closedAt: timestamp("closed_at"),
totalAmount: numeric("total_amount", { precision: 10, scale: 2 }),
totalPaid: numeric("total_paid", { precision: 10, scale: 2 }),

```

5.2 — Nuevos endpoints (server/routes.ts)

POST	/api/spa/accounts/:id/items	→ Agregar ítem al folio
DELETE	/api/spa/accounts/:id/items/:itemId	→ Eliminar ítem del folio
POST	/api/spa/accounts/:id/payments	→ Registrar pago (incluye anticipos)
GET	/api/spa/accounts/:id/payments	→ Listar pagos del folio
POST	/api/spa/accounts/:id/close	→ Cerrar folio con receiptType
GET	/api/spa/accounts/:id/summary	→ Resumen: ítems + pagos + saldo

Lógica del endpoint /close :

- Validar que `totalPaid >= totalAmount` (saldo cero o a favor).
- Si hay saldo pendiente, retornar error con el monto adeudado.
- Si `method = "room_charge"`, crear un cargo en la tabla `charges` de la reserva indicada con descripción "SPA - [detalle]".
- Marcar `spaAccount.status = "closed"` y guardar `closedAt`.

5.3 — Cambios en el frontend (client/src/pages/spa.tsx)

Modal de folio de turno (reemplaza el cierre simple actual):

El modal debe tener dos secciones:

Sección izquierda — Cargos:

- Lista de ítems del folio con descripción, cantidad, precio unitario y subtotal.
- Botón "Agregar cargo" que abre un sub-formulario con:
 - Tipo: Tratamiento / Producto / Servicio extra / Consumo (ej: pileta, toalla, bebida)
 - Descripción libre
 - Precio unitario
 - Cantidad
- Opción de eliminar ítems mientras el folio está abierto.
- Subtotal, impuestos (si aplica) y **total a cobrar**.

Sección derecha — Pagos:

- Lista de pagos ya registrados (incluyendo anticipos con etiqueta "SEÑA").
- Botón "Registrar anticipo" (disponible desde la creación del turno): guarda un pago con `isAdvance: true`.
- Al cerrar: botón "Agregar pago" con método de pago (efectivo, débito, crédito, transferencia, MercadoPago, cargo a habitación).
 - Si elige "cargo a habitación": mostrar buscador de reservas activas por número de habitación o nombre del huésped.
- **Saldo pendiente** calculado automáticamente: `total - suma de pagos`.
- Solo habilitar el botón "Cerrar folio" cuando saldo = 0.

Selección de comprobante al cerrar:

- Dropdown: Ticket / Factura A / Factura B / Factura C / Nota de Crédito.

Anticipos desde la agenda:

- Al crear un turno, agregar campo opcional "Registrar seña" con monto y método de pago.
- Si se ingresa, crear automáticamente la `spaAccount` y registrar el pago con `isAdvance: true`.
- El turno debe mostrar un ícono o badge "SEÑA" en la grilla de la agenda.

Notas para Replit

- Ejecutar las tareas **en el orden indicado** (1 → 5) ya que hay dependencias entre ellas.
- En la Tarea 5, el schema cambia antes que el backend y el frontend. Actualizar `shared/schema.ts` primero y verificar que el build no rompa nada antes de continuar.
- El MemStorage (`server/storage.ts`) necesita actualizarse en paralelo con el schema para que los nuevos campos funcionen en desarrollo.
- No es necesario migrar a PostgreSQL real en esta etapa. Todos los cambios deben ser compatibles con MemStorage.
- Mantener la nomenclatura existente (camelCase en TypeScript, snake_case en SQL).
- Los textos de interfaz deben estar en español, consistente con el resto del sistema.

Documento generado para el proyecto Maran Suite System — Módulo SPA

Próximo módulo a especificar: según prioridad operativa acordada